Alibaba AI Research Scheme 2026

Research Proposal

Project Name	Automated Production of MCP-Endpoint Expert Models via Cold-Start Distillation for AIOps Fault Response
Topic	Topic 1 (Large–Small Model Collaboration) / Topic 10 (AIOps)
Date	March 2026
Organization	*<FILL: University/Institution Full Name>*
Author (PI)	*<FILL: PI Full Name>*
Email	*<FILL: pi@university.edu>*

Submitted in response to Alibaba AI Research Scheme 2026.
Funding: RMB 600,000. Duration: One year.

1. Introduction

1.1 Title of Project

Automated Production of MCP-Endpoint Expert Models via Large-Model Cold-Start Distillation for AIOps Fault Emergency Response

1.2 Background and Significance

Industrial motivation. Large-scale internet platforms such as Alibaba operate complex, heterogeneous infrastructure that must maintain near-zero downtime. Fault-emergency response (AIOps) is therefore a critical operational capability, yet it remains highly labor-intensive: on-call engineers must diagnose alerts, correlate logs across dozens of subsystems, and prescribe remediation actions under severe time pressure. The Qwen3 model family [Alibaba Cloud, 2025] and the broader maturation of Large Language Models (LLMs) open the door to automating this workflow. However, a single cloud-hosted LLM incurs API latency of 500 ms–2 s per call—and can time out entirely during high-volume fault storms—whereas locally deployed small language models (SLMs) respond in <100 ms, making them indispensable for latency- sensitive AIOps execution.

The hidden assumption in LLM–SLM collaboration research. Topic 1 of this scheme asks how large and small models can collaborate effectively. Existing collaboration frameworks (e.g., FrugalGPT, RouteLLM) implicitly assume that *specialist small models already exist*: their contribution is an efficient routing or scheduling policy that decides which model handles a given query. They do not answer the more fundamental question: *where do specialist small models come from?*

Reasoning-chain training as the established path to capable SLMs. A separate line of work—STaR [Zelikman et al., 2022], Distilling Step-by-Step [Hsieh et al., 2023], and related methods [Gu et al., 2024, Dong et al., 2024, Agarwal et al., 2024]—has convincingly demonstrated that equipping an SLM with *chain-of-thought (CoT) reasoning* during training, rather than merely memorising input–output pairs, dramatically improves its performance on domain-specific tasks. The bottleneck is therefore data: obtaining high-quality CoT annotations requires either expensive human annotators or a large teacher model that can generate rationales on demand. Neither resource is available at *cold-start*, i.e., when a team launches a new business domain with no labelled data and no pre-existing expert model.

The research gap. As illustrated in Figure 1, existing SLM-production methods all require at least one of: (a) human-labelled CoT, (b) a teacher model with prior knowledge of the domain, or (c) an already-performing student model to bootstrap from. In cold-start settings—precisely the condition that governs new AIOps sub-scenarios— none of these prerequisites hold. This gap motivates our core proposal: an automated pipeline that generates CoT-annotated training data *from raw* $\langle$*input, output*$\rangle$ *pairs alone*, requiring no human intervention and no domain-aware teacher.

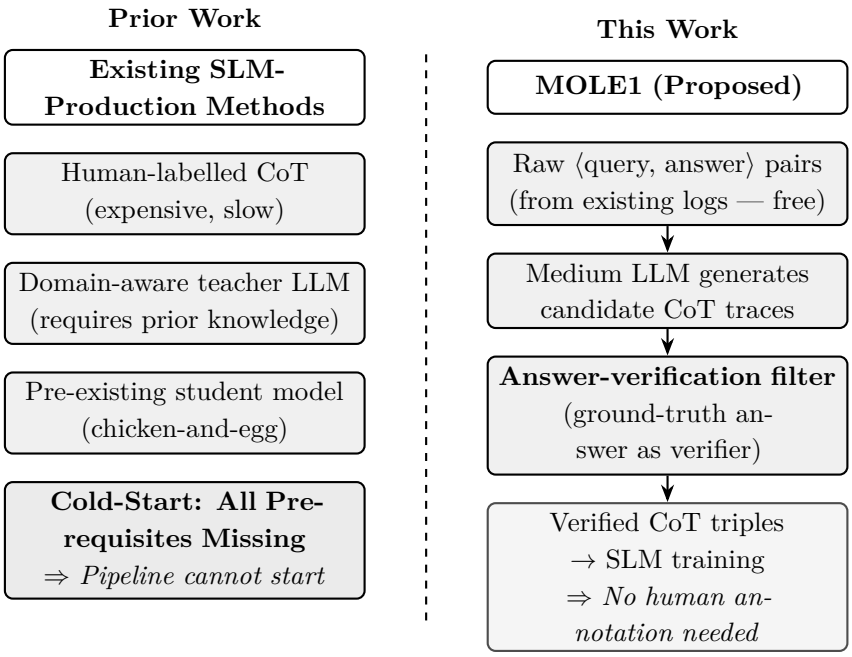

Figure 1: Existing SLM-production methods fail at cold-start (left); MOLE1 succeeds by using the observed answer as an automatic verifier (right).

Why AIOps is the ideal proving ground. Three structural properties make AIOps the best domain in which to validate the proposed approach.

1. **Data abundance without labelling cost.** Every large-scale platform accumulates rich ⟨alert+log, remediation-action⟩ pairs in its operational databases—precisely the ⟨input, output⟩ format MOLE1 requires—with zero additional annotation effort.

2. **Objective, binary verification.** "Did the proposed remediation resolve the fault?" is a ground-truth signal available in historical records, enabling fully automated offline evaluation via incident replay—no live environment needed.

3. **Quantifiable latency advantage.** Fault response is acutely latency-sensitive. A locally deployed SLM responds in <100 ms; a cloud LLM API call costs 500 ms–2 s and times out under peak-load fault storms. This 10–20× gap is a concrete, measurable industrial pain point that MOLE1 directly addresses.

4. **Strategic dual coverage.** The proposed method directly addresses both Topic 1 (LLM–SLM collaboration mechanism) and Topic 10 (fault emergency response AIOps), demonstrating that a principled methodology can simultaneously solve two industrial pain points.

1.3 Objective of Project

This project pursues four tightly coupled objectives:

O1. MOLE1 pipeline design. Design and implement the full cold-start pipeline for automated CoT generation and SLM distillation, including the answer-verification filtering mechanism.

O2. AIOps validation. Validate the pipeline on real or realistic AIOps datasets, demon-

strating that domain-specialist SLMs produced by MOLE1 match or outperform general LLMs on fault diagnosis and remediation benchmarks.

O3. MCP encapsulation. Package each trained SLM as a standardised Model-Context-Protocol (MCP) tool endpoint, enabling plug-and-play integration with a cloud LLM Planner layer.

O4. Specialisation mechanism analysis. Provide rigorous empirical and theoretical analysis explaining *why* a domain-focused SLM can outperform a general LLM on targeted tasks (the "specialisation beats generalisation" phenomenon).

O5. Cross-domain transfer study. Validate MOLE1's domain generality by applying the identical pipeline to at least one additional domain beyond AIOps (e.g., code-review classification or customer-service triage), demonstrating that the method's value is not scenario-specific.

1.4 Development Method

Core Insight: MOLE1—Answer-Verified CoT Synthesis

We propose **MOLE1** (**M**odel-**O**utput-**L**abel-**E**licited CoT Synthesis, iteration 1), a domain-agnostic pipeline that converts raw ⟨input, output⟩ pairs into CoT-annotated training data *without any human annotation or domain-aware teacher model.* The key innovation is using the *known answer as an automatic verifier*:

$$\underbrace{\langle \text{query, answer} \rangle}_{\text{available cold-start}} \xrightarrow{\text{medium LLM, } k \text{ samples}} \text{candidate CoTs} \xrightarrow{\text{answer-consistency filter}} \underbrace{\langle \text{query, CoT}^*, \text{answer} \rangle}_{\text{verified triples}} \xrightarrow{\text{SLM fine}}$$

A medium-sized model (e.g., Qwen3-7B) generates k candidate reasoning chains per query via temperature sampling (τ=0.8). Only chains whose final conclusion matches the ground-truth answer are retained—a pass/fail criterion that requires *no domain knowledge.* The verified triples fine-tune a compact SLM (e.g., Qwen3-1.5B/3B) exclusively on the target domain.

Domain generality. Although we validate MOLE1 on AIOps, the pipeline is *entirely domain-agnostic*: it requires only that (i) historical ⟨input, output⟩ pairs exist and (ii) answer correctness can be verified automatically. This condition holds across a wide range of industrial scenarios—customer-service triage, code-review classification, supply-chain anomaly detection—making MOLE1 a general-purpose cold-start engine for LLM–SLM collaboration, well beyond its AIOps instantiation. The AIOps domain is chosen as the primary validation testbed because it uniquely satisfies all three structural advantages described above and provides a clear, reproducible experimental protocol.

Four-Stage Pipeline

Figure 2 shows the complete MOLE1 pipeline.

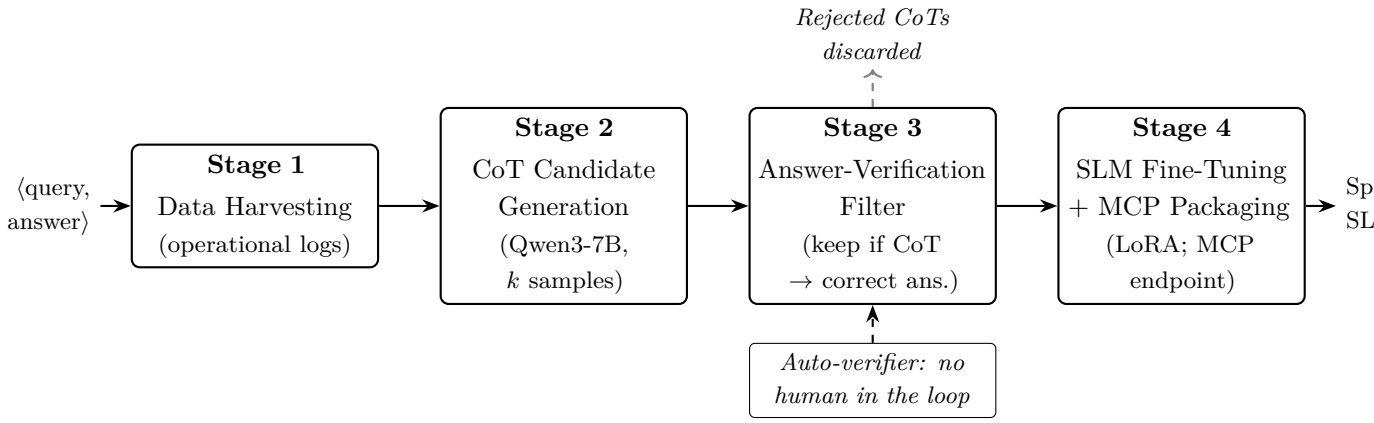

Figure 2: MOLE1 four-stage pipeline for automated SLM production from cold-start data.

Stage 1 – Data Harvesting. Collect ⟨alert+log, remediation⟩ pairs from operational logs. No manual labelling; records come from existing incident-management systems.

Stage 2 – CoT Candidate Generation. For each query, Qwen3-7B generates k candidate reasoning chains (temperature τ=0.8, top-p=0.95). This step requires no domain knowledge on the part of the generating model.

Stage 3 – Answer-Verification Filtering. Each candidate CoT is evaluated: does the final conclusion match the recorded remediation action? Matched CoTs are retained; mismatched ones are discarded. Acceptance rate varies with task complexity; analogous filtering in STaR [Zelikman et al., 2022] yields 30–70% retention under comparable settings. We note that data quality is a prerequisite: records involving operator error or mislabelled remediation are excluded via a lightweight anomaly filter in Stage 1.

Stage 4 – SLM Fine-Tuning and MCP Packaging. The verified triples train a compact SLM via supervised fine-tuning (SFT) with LoRA. The trained model is wrapped as an MCP-compliant tool endpoint and registered with the Planner layer.

Experimental Design and Evaluation Protocol

Dataset. We use a combination of (a) a proprietary AIOps dataset from Alibaba operational logs (subject to a mutually agreed data-sharing protocol with Alibaba's SRE team) and (b) the publicly available AIOps-2022 benchmark [Notaro et al., 2021] for reproducibility. Experiments cover three AIOps sub-scenarios: database-layer faults, network partition events, and container resource exhaustion. A cross-domain transfer experiment on a public code-review dataset will validate MOLE1's domain generality (Objective O5).

Baselines.

- **LLM-only**: Qwen3-72B queried via API, no local SLM.
- **SLM w/o CoT**: SLM fine-tuned on ⟨input, output⟩ pairs only (no reasoning chains).
- **SLM + teacher CoT**: SLM fine-tuned with CoT generated by Qwen3-72B acting as teacher (the "oracle" upper bound requiring a capable teacher).

- **MOLE1 (ours)**: SLM fine-tuned on answer-verified CoT from Qwen3-7B.

Metrics. Remediation-action accuracy (top-1 match rate), inference latency (p50/p99), and compute cost per query (FLOPs / API tokens). Statistical significance assessed via bootstrap resampling (n=1000).

System Architecture: Planner + MCP Tool Layer

Figure 3 depicts the overall deployment architecture.

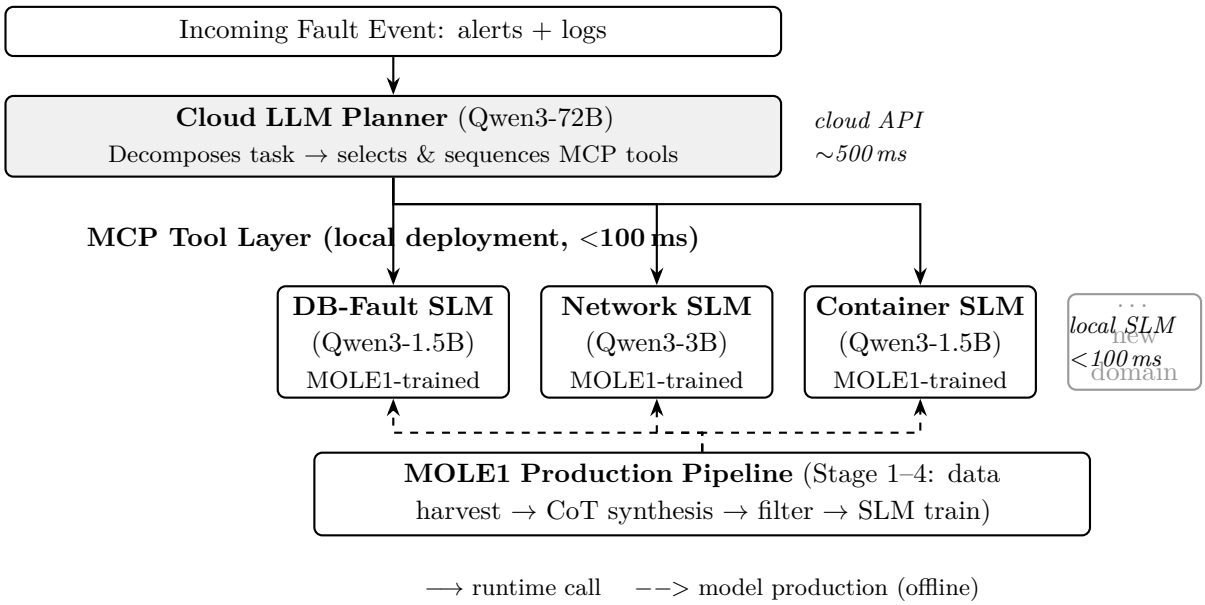

Figure 3: System architecture: cloud LLM as Planner orchestrates locally deployed MCP-endpoint SLMs produced by the MOLE1 pipeline.

The cloud-hosted LLM (Qwen3-72B or equivalent) acts as a **Planner**: it decomposes incoming fault events into sub-tasks, determines execution order, and invokes the appropriate MCP tool endpoints. Each MCP tool wraps an SLM that has been trained exclusively for one AIOps sub-scenario (e.g., database slowdown diagnosis, network partition detection, container OOM triage).

The Planner–Executor split reflects the asymmetric nature of the two roles:

- The Planner requires *global semantic understanding* to decompose novel, multi-step fault events—a capability that demands the breadth of a large model.
- Each Executor requires *deep, narrow expertise* in a single sub-domain— a capability that benefits from focused training on domain-specific CoT data, and which does *not* benefit from the generalisation enforced by multi-task training on a large model.

This design also delivers the latency advantage: only the Planner call traverses the cloud API (∼500 ms); all Executor SLM inferences run locally (<100 ms), so the end-to-end system responds in seconds even under high alert volume.

Why a Specialist SLM Can Outperform a General LLM

The "specialisation beats generalisation" phenomenon is well-supported by the distillation literature [Hsieh et al., 2023, Dong et al., 2024] and follows from two observations: (i) a general LLM's parameters are shared across thousands of tasks, diluting per-domain capacity; (ii) an SLM trained exclusively on domain-specific CoT data develops tightly aligned internal representations for that domain, achieving higher precision despite far fewer parameters. We will provide both empirical evidence (ablation studies) and a capacity-allocation analysis to formalise this intuition.

2. Milestone

2.1 Duration of Project

One year, commencing from the effective date of the Research Collaboration Agreement.

2.2 Plan

Table 1: Project milestones and deliverables.

Stage	Activities	Deliverable(s)	Period
M1	**Pipeline Design & Data Preparation.** Formalise MOLE1 algorithm; collect and pre-process AIOps ⟨input, output⟩ dataset; implement CoT candidate generation (Qwen3-7B) and answer-verification filter.	Dataset & filter module; technical report.	Month 1–3
M2	**SLM Training & Benchmarking.** Fine-tune SLMs on verified CoT triples; conduct comparison experiments against general LLM baselines on AIOps benchmarks; specialisation-mechanism analysis.	Trained SLM models; experiment report; draft of CCF-A paper #1.	Month 4–7
M3	**MCP Packaging & System Integration.** Wrap each SLM as an MCP tool endpoint; integrate Planner (Qwen3-72B) with MCP tool layer; end-to-end latency and accuracy evaluation.	Open-source MCP toolkit; system demo; draft of CCF-A paper #2.	Month 8–9
M4	**Evaluation, Patent Filing & Final Deliverables.** Full system evaluation; user study with Alibaba stakeholders; patent applications; final project report.	Final report; published/submitted papers; patent applications (1–2).	Month 10–12

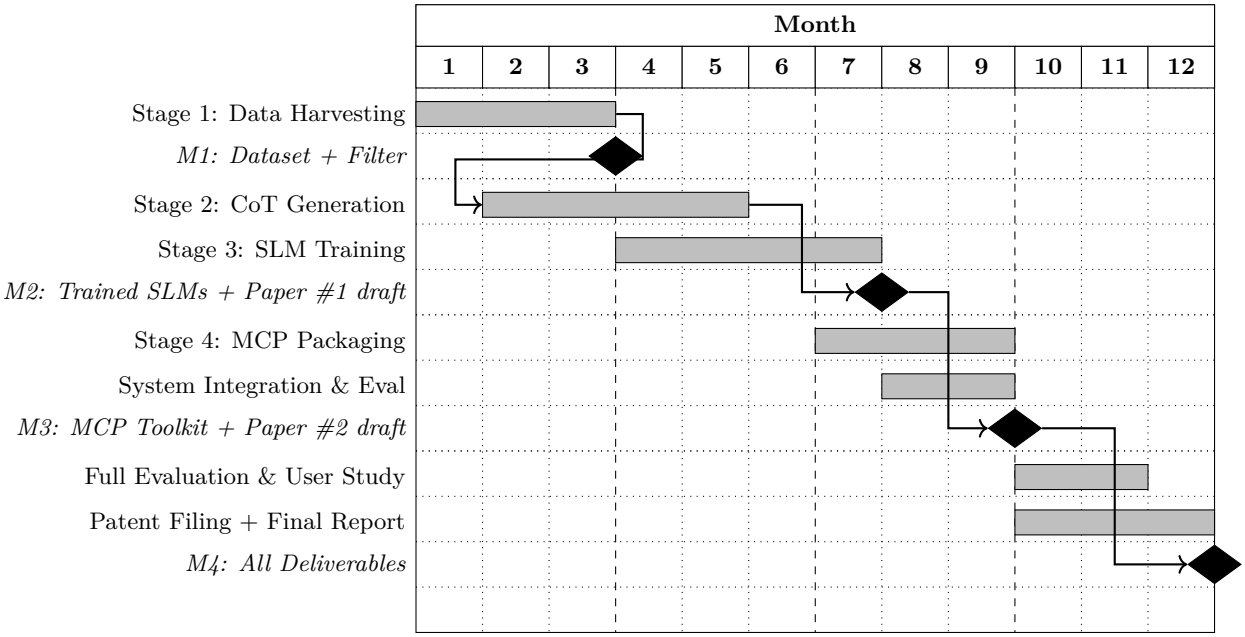

Figure 4: Project Gantt chart across four milestones.

3. Deliverable(s)

The following final deliverables are committed for this project.

Description	Mode of Delivery	Remarks
Prototype system: MOLE1 cold-start SLM production pipeline	Source code (open-source)	Reproducible; released on GitHub
MCP-compliant AIOps tool-endpoint toolkit	Source code + documentation	Compatible with standard MCP protocol
2–3 academic papers on MOLE1 methodology and AIOps evaluation	Academic papers	Targeting CCF-A or equivalent top venues (e.g., NeurIPS, ICML, ACL, ICSE, FSE)
1–2 invention patents on cold-start CoT synthesis and MCP packaging	Patent applications	International or China patents
Interim reports (kick-off, mid-term, final)	Project reports	Submitted per agreed schedule

Expected technical improvement: SLMs produced by MOLE1 are expected to achieve accuracy within 5% of GPT-4-class LLMs on AIOps fault-diagnosis benchmarks while reducing per-inference latency by $>10\times$ (from $\sim$1 s to $<$100 ms) and token cost to zero (local inference).

Expected business value for Alibaba: A reusable, domain-agnostic pipeline that Alibaba SRE teams can apply to any new AIOps sub-scenario within days, without labelling budget or pre-existing expert models.

4. Human Resources, Equipment and Budgetary Plan

4.1 Personnel

Name	Profile	Expense (RMB/mo.)	Pe
<FILL: PI Name>	*<FILL: Professor>*, *<FILL: University Name>*	12,000	3 mo
<FILL: PhD Student 1 Name>	PhD Student, *<FILL: University Name>*	12,000	12 mo
<FILL: PhD Student 2 Name>	PhD Student, *<FILL: University Name>*	12,000	12 mo
			Sub-total

Note: Internship periods are indicative; exact dates subject to Alibaba SRE team schedule.

4.2 Equipment & Supplies

Item	Unit Price (RMB)	Total (RMB)	Remarks
GPU cloud computing (A100 node-hours)	—	91,000	For SLM training runs (Stage 2–3)
Local inference server (lease/purchase)	50,000	50,000	MCP endpoint deployment & latency testing
Conference travel (2 venues)	15,000	30,000	Domestic + international top-venue presentation
Open-access publication fees	10,000	20,000	ACL Anthology / NeurIPS open access
Sub-total Research Expenses (b)		**191,000**	

4.3 Budgetary Plan

Item	Remarks	Amount (RMB)
(a) Total Personnel Expenses	See Section 4.1	324,000
(b) Total Research Expenses	See Section 4.2	191,000
(c) Indirect Cost	University overhead: 16.5% × (a + b) = 16.5% × 515,000 ≈ 85,000	85,000
Grand Total	(a) + (b) + (c)	**600,000**

5. Biographical Sketches of Principal Investigator(s)

<FILL: PI Name>
<FILL: Professor / Associate Professor>, *<FILL: Department Name>*, *<FILL: University Name>*
Email: *<FILL: email>* Homepage: *<FILL: homepage URL>*

Research Interests: Large Language Models, Knowledge Distillation, AIOps, Intelligent Software Systems

Representative Publications (recent 5 years):

1. *<FILL: Author(s), Title, Venue, Year>*
2. *<FILL: Author(s), Title, Venue, Year>*
3. *<FILL: Author(s), Title, Venue, Year>*

Grants and Awards:

- *<FILL: Funding Source, Project Title, Amount, Year>*
- *<FILL: Funding Source, Project Title, Amount, Year>*

Relevant Industrial Experience: *<FILL: Relevant industrial collaboration experience, if any>*

6. References

6. References

R. Agarwal, N. Vieillard, P. Stanczyk, S. Ramos, M. Geist, and O. Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations (ICLR)*, 2024.

Alibaba Cloud. Qwen3 technical report. Technical report, Alibaba Group, 2025.

J. Dong et al. DDK: Distilling domain knowledge for efficient large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Y. Gu, L. Dong, F. Wei, and M. Huang. MiniLLM: Knowledge distillation of large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

C.-Y. Hsieh, C.-L. Li, C.-K. Yeh, H. Nakhost, Y. Fujii, A. Ratner, R. Krishna, C.-Y. Chen, and T. Pfister. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Findings of the Association for Computational Linguistics (ACL)*, 2023.

A. Notaro, J. Cardoso, M. Gerndt, and M. Brenner. Aiops: Real-world challenges and research innovations. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 12(6), 2021.

E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.